

开锁判责方案设计

🔍 | 📄 正文 1.3 万字 (23.5 页) | 浏览 26 次 (1 人) | 全部 >

AI 速览

背景： 开锁换锁行业存在高客诉率、售后表现差、平台赔付成本高、舆论风险大等问题，同时监管部门对高危行业要求趋严。大连被选为试点城市，验证强监管治理策略的有效性。

冲突：

- 行业问题集中：** 大连开锁换锁行业客诉率高达6.8% (全国平均2.6%)，单均赔付1.8元 (全国1.3元)，存在暴力开锁、不合理收费、飞单等违规行为。
- 监管压力：** 央视315曝光类似事件，平台需未雨绸缪加强商户监管。
- 技术挑战：** 需实现交易/预约/履约线上化，处理高并发订单消息（尾款支付触发判责进件）。

问题：

如何通过技术手段实现高效、精准的判责，降低客诉率和赔付成本，同时满足监管要求？

解决方案：

1. 判责系统设计

- 进件来源：**
 - 系统巡检：** 尾款支付订单自动进件（大连城市+开锁类目校验）。
 - 太平洋工单：** 客诉工单自动匹配FAQ（如“线下交易找商户”“不合理收费”）。
 - 手动发起：** 运营人员输入订单号+选择问题场景（暴力开锁/不合理收费/飞单）。
- 判责规则：**
 - 暴力开锁/飞单：** 直接判定重责。
 - 不合理收费：** 按加价区间分级（ ≤ 1.2 倍轻责，1.2-1.5倍中责， > 1.5 倍重责）。
- 流程：** 初判→商户申诉（48小时）→复判→完成，全流程留痕。

2. 技术实现

- 消息队列：** Kafka处理尾款支付事件，策略模式支持多行业扩展。
- 延迟队列：** 48小时申诉超时自动流转复判。
- 分布式锁：** Redis防重复进件，确保幂等性。
- 分单调度：** 实时分单+工单池定时任务，轮询分配判责人员。
- 监控告警：** Cat监控工单状态、超时率，异常实时告警。

3. 数据与权限

- 数据库：** MySQL分库分表（按业务类型），读写分离。
- 缓存：** Redis缓存工单信息，提升查询性能。
- 权限：** 新增行业运营权限，隔离判责数据。

4. 试点目标

- 治理效果：** 2025年底万单服务率下降50%，完整履约率达50%。
- 扩展性：** 模式可复制至管道疏通、防水补漏等高危行业。

上线时间： 8月31日前完成大连试点部署。

开锁换锁行业在大连试点的治理目标是什么？

开锁判责系统中针对不合理收费的判责标准是如何划分的？

🖋 风格切换

💬 继续提问

📄

🔄

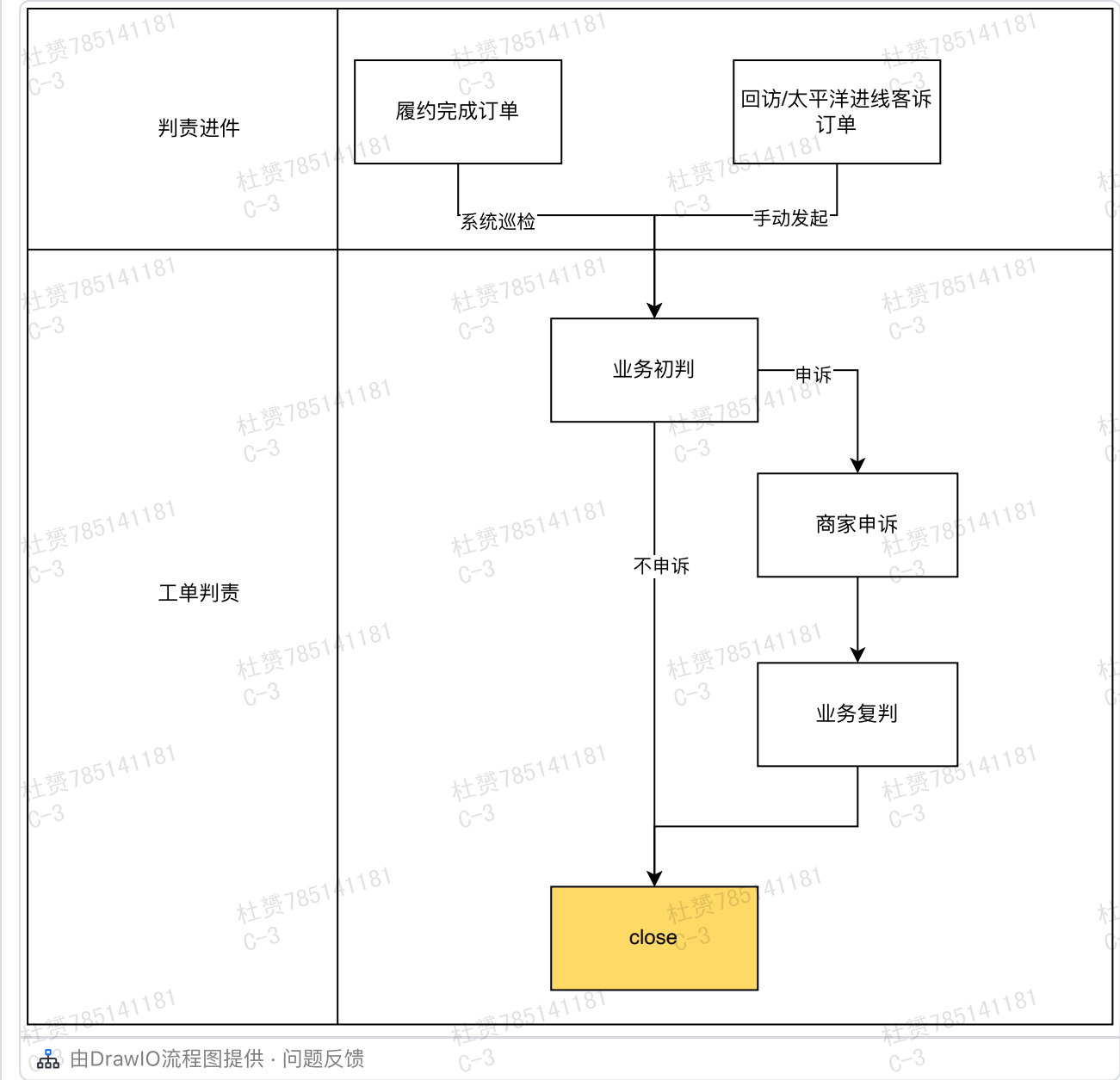
👍

👎

...

×

需求名称：开锁判责	优先级：P0
背景	背景：开锁换锁行业中履约表现较差，具体表现在高客诉率、售后表现差、平台赔付成本高、舆论风险大等特性；同时，各地监管部门对于生活服务高危行业正在快速收严要求，同时也在提高对平台的要求；已筛选大连作为试点开锁换锁履约强监控试点城市，验证治理策略的有效性。 [PRD]开锁治理大连试点MVP需求-交易/预约/履约线上化改造
求收益	1、开锁行业易产生舆情，对平台商誉影响大 ✎ 美团你能不能做点好事啊？我的周五晚上就这么被开锁套路毁了…… 阅读量1.2万 2、开锁换锁履约差在商户层面有集中性，通过强监管降低完单服务率，并降低单均赔付成本（大连单均赔付1.6元/单 vs 全国1.3元/单）； 3、开锁行业若能跑通强监管模式，后期可扩展至管道疏通和防水补漏等高危行业，判责可扩展性强。
判责方案	判责标准（仅管控过度维修、不合理收费） 1、针对试点城市大连，有完整履约（以有尾款回收为衡量标准）订单，单单发起判责； 支持人工发起的规则：待补充 - 输入订单号 - 选择问题来源：太平洋进线（三级FAQ为：生活服务-线下交易找商户、疑似飞单、加价接待、商户价格虚高、消费结果不满意）、回访、履约数据、社媒/舆情 - 选择工单问题：过度维修/暴力开锁、不合理收费、飞单 2、有责类型分为：暴力开锁、不合理收费、飞单三类；无履约信息，归类到【无法判责】； 3、有责程度标签为：轻责、中责、重责； 4、判责页面交互可参照无忧通 附： 1、判责流程图



2、有责标准（判责采集信息为：维修前后照片）

类目	违规FAQ	二级细分场景	责任区间	有责程度
开锁换锁（大连）	过度维修/暴力开锁	 无	不区分	重责
	不合理收费	锁具等级与技术开锁收费标准不符合	≤1.2倍	轻责
			1.2倍<X≤1.5倍	中责
			> 1.5倍	重责
		白牌锁具价格高于约定价格	≤1.2倍	轻责
			1.2倍<X≤1.5倍	中责
			> 1.5倍	重责
		品牌锁具价格高于电商旗舰店同款价格	≤1.2倍	轻责

		1.2倍<X≤1.5倍		中责
		> 1.5倍		重责
	飞单	無	不区分	重责
	无违规	-	-	-
	无法判责	无完整履约信息	-	-
预计上线时间	8月31日前			

业务PRD

1 需求综述

1.1 需求背景

开锁换锁行业中履约表现较差，具体表现在高客诉率、售后表现差、平台赔付成本高、舆论风险大等特性；同时，各地监管部门对于生活服务高危行业正在快速收严要求，同时也在提高对平台的要求；已筛选大连作为试点开锁换锁履约强监控试点城市，验证治理策略的有效性。目前开锁业务只在大连展开，后续要进行开城，判责操作也需要做适配

[PRD]开锁治理大连试点MVP需求-交易/预约/履约线上化改造

1试点背景

- 
1.

各地监管部门对于生活服务高危行业正在快速收严要求，同时也在提高对平台的要求。为保障消费者权益，平台需要未雨绸缪增强对商户的监管，丰富治理手段。
2.

央视315曝光啄木鸟事件体现出平台在的财收结构、商家结构组成较为脆弱。头部客户一旦出问题，大量CPC停投、商户关店避风头，业绩会受到较大打击，且没有应对方案。
3.

本方案将通过城市试点的方式，针对开锁换锁行业中履约表现较差的城市进行专项治理，验证治理策略的有效性。

1.试点行业：开锁换锁（生服上门中万单服务率最高类目）

2.开锁履约问题分析：

- 投诉率高：过去一年，开锁换锁行业在生服上门全行业中万单服务率最高，存在履约质量不稳定、客诉率高等问题。
- 解决率差：线下飞单严重，脱离平台交易，平台无法赔付全款，用户接受度较低。
- 高频客诉问题：开锁换锁行业主要问题场景集中在线下交易、履约结果不满意、退款纠纷等，大连市客诉场景基本符合行业现状。在这些高频场景下，存在问题集中的大连在线商家，蚂蚁开锁、老兵、大连便民开锁换锁、实诚小伙等具有典型问题集中性。其中蚂蚁开锁涉及1例违法犯罪高危场景，用户不认可商家报价后，接到商家频繁电话骚扰近30通并威胁用户，用户报警。
- 赔付成本高：大连市开锁换锁行业在2024-05至2025-04的财务收入为1,523,956.57元，赔付总额为**31,327**元，单均赔付1.8元，高于行业平均水平（1.3元），赔付成本占财务收入的2%。
- 商户集中性：2025年3月至5月，全国开锁换锁订单量在100单以上的商家，按照品牌维度统计万单服务率高于行业平均水平的门店数量，问题门店数量最多的品牌与其问题门店数量所占比也同为行业内最高值，体现了履约不佳商户的品牌集中性。其中，大连市过往一年产生订单交易的开锁商户数量为**61**家，截止2025-05-31，仍在线营业的商户数量为**27**家，在线商家的履约客诉表现如下，实诚小伙、福顺达、嘎嘎快、老兵这4家门店过往客诉率较高，其中实诚小伙、老兵开锁、诚信老兵、大连便民开锁4家门店命中违规，已缴纳保证金。

3.试点城市选择：大连市

针对2024年5月至2025年4月各城市开锁换锁行业履约表现，限定客诉数量>1000，对客诉率进行倒序排列，可得下表，取客诉率排名top1城市作为治理试点城市。

✓ 附录: 开锁履约不佳城市top5

1 试点目标

1.治理目标

通过试点效果和行业逐步推全，试点城市开锁换锁行业实现：

	万单服务率	备案资质覆盖率	完整履约率（有预约、派单、师傅信息、履约留痕信息、尾款结算的完整流程收费订单量占总订单比例）
指标	到25年底下降50%	到25年底覆盖60%；到26年中全覆盖	到25年底达到50%

2.验证对供给覆盖、财收会造成多大影响

拓城方向：根据各城市开锁行业管理办法调研结果，大部分公安协助意愿高的城市，开锁客诉率均在1%以下。除大连外，公安配合意愿高的城市为沈阳（客诉率2.3%）、长春（客诉率3.1%）、合肥、长沙、贵阳、乌鲁木齐、呼和浩特，可作为拓城优先选择。

1 需求目标&价值

- 1) 开锁换锁履约差在商户层面有集中性，通过强监管进行差商管理、降低万单服务率（到25年底下降50%），并降低单均赔付成本。
- 2) 开锁行业若能跑通强监管模式，后期可拓展至管道疏通和防水补漏等高危行业，判责可扩展性强。

2 需求内容

<https://customer-complaint-evaluator.mynocode.host>

功能点	逻辑	图示
判责进件	1、尾款支付订单 进件逻辑： • 订单对应的团单分类为：二级团单类目“开锁” • 订单核销门店的城市为大连lion配置 • 1、进件逻辑：订单完成尾款支付后，实时进件（实现：监听交易侧尾款支付消息，当交易侧尾款支付成功后会向mafka相对应的topic发送一条消息，包含订单id、服务人员id）进件插入住数据表、attr表 • 2、分单逻辑：先实时派单，如果是在工作时间内，那么直接派单，并插入判责单log、record、pool表；如果不是在工作时间内，插入log、pool表，后续通过定时任务扫描pool表进行分单 问题场景： • 尾款支付订单进件的问题场景“不合理收费” （开锁尾款支付进件上线后发现订单的mq消息量太大，导致系统异常，准备先使用T+1去生成判责单。）	

2、太平洋工单

- 工单关联的门店的城市为大连，门店类目为开锁换锁 门店id类目 826
- 工单有关联的订单信息
- 工单的三级FAQ为：生活服务-线下交易找商户、线下交易找商户、疑似飞单、加价接待、商户价格虚高、消费结果不满意（lion配置）
- 1、进件逻辑：太平洋生成工单的同时向对应topic `csc-pacific-send-daocan-mq` 发送一条消息（包含太平洋工单信息），解析消息并且验证上面的条件；使用的策略模式，目前判责的行业有保洁、洗涤、无忧通、维修，本次扩开锁，使用策略模式可以以后扩行业更好的扩展（开锁判责这块几乎都使用了策略模式，包括进件逻辑处理、分单逻辑处理、不同行业的处理器等等 ps可以后续补充）
- 2、和尾款支付订单大致相同，不过两者做的类不同，一个是专门为太平洋服务的，一个新写的专门为开锁的尾款支付订单服务的

```
ORDER_ID( attrKey: "orderId", desc: "订单id"), 2个用法
DP_SHOP_ID( attrKey: "dpShopId", desc: "点评商户id"), 3个用法
WORK_ORDER_CREATE_TIME( attrKey: "workOrderCreateTime", desc: "工单创建时间"), 1个用法
CLAIM_TYPE_DESC( attrKey: "claimTypeDesc", desc: "工单问题描述"), 1个用法
CLAIM_TYPE( attrKey: "claimType", desc: "工单问题类型, 6级FAQ ID"), 11个用法
CLAIM_TYPE_EXPLAIN( attrKey: "claimTypeExplain", desc: "6级FAQ id解释"), 1个用法
CLAIM_MATERIAL( attrKey: "claimMaterial", desc: "工单问题材料"), 1个用法
PACIFIC_WORK_ORDER_ID( attrKey: "pacificWorkOrderId", desc: "太平洋工单id"), 2个用法
PACIFIC_WORK_ORDER_OPERATOR_NAME( attrKey: "pacificWorkOrderOperatorName", desc: "太平洋工单操作人姓名"),
PACIFIC_WORK_ORDER_OPERATOR_MIS( attrKey: "pacificWorkOrderOperatorMis", desc: "太平洋工单操作人mis号"),
PACIFIC_REMARK_RESULT( attrKey: "pacificRemarkResult", desc: "太平洋工单坐席备注的处理结果"), 1个用法
PACIFIC_SELECT_RESULT( attrKey: "pacificSelectResult", desc: "太平洋工单坐席选择的处理方案"), 1个用法
LIABILITY_PARTY( attrKey: "liabilityParty", desc: "责任方"), 1个用法
```

问题场景和FAQ映射关系

FAQ	问题场景
生活服务-线下交易找商户、线下交易找商户、疑似飞单	飞单
加价接待、商户价格虚高、消费结果不满意	不合理收费

3、重复进件判断

如果订单已有生成的工单，则拦截进件

重复判断逻辑：在进件时，根据问题场景，判断对应订单是否已进件。若已进件，则不重复进件。但需要在原订单上也打上新的来源标记“太平洋工单”（展示为标签，只给运营侧展示）

	1、筛选器 • 时间： 创建时间 • 搜索： 订单号、工单号 与 洗涤 • 问题分类以及对应问题场景： 售后体验问题-过度维修/暴力开锁、售后体验-不合理收费、红线问题-飞单 • 工单来源： 系统巡检、太平洋工单、回访、差评、履约数据、社媒/舆情 • 城市： 可筛选对应城市下的门店判责单 2、工单列表 状态筛选器：全部、待初判、待复判、初判中、商户申诉中、复判中、已完成 • 状态 • 判责人员 • 工单编号 • 订单详情 • 判责对象 • 工单来源 • 问题场景： 巡检或太平洋进件后，在初判之前为“未归类” • 判责结果 • 创建时间 • 操作	• 订单号 1379138 • 工单来源 ☐ 系统巡检 ☒ 太平洋工单 ☐ 回访 ☐ 差评 ☐ 履约数据 ☐ 社媒/舆情 • 工单问题 ☒ 过度维修/暴力开锁 ☐ 不合理收费 ☐ 飞单 对应判责结果 ☐ 轻责 ☐ 中责 ☒ 重责 问题描述 请输入问题描述（选填） 取消 确认发起
海星平台-手动发起工单	判责列表新增一个“手动发起工单”的功能，放在“导出数据”旁边 1、输入订单号 （校验订单号是否合规） 2、选择“工单来源” 系统巡检、太平洋工单、回访、差评、履约数据、社媒/舆情 3、选择“工单问题” 过度维修/暴力开锁、不合理收费、飞单 4、问题描述 非必填，支持输入文本 5、操作按钮 • 取消： 关闭弹窗 • 确认发起： 点击后创建新工单	

- 点击“确认发起”，校验该订单下是否已经有相同**问题场景**的工单问题，如已有，则提示“此订单已有相同问题的工单，请勿重复发起”

* 订单号

1379138

* 工单来源

☐ 系统巡检

☒ 太平洋工单

☐ 回访

☐ 差评

☐ 履约数据

☐ 社媒/舆情

* 工单问题

☐ 过度维修/暴力开锁

☒ 不合理收费

☐ 飞单

* 不合理收费的细分场景

☐ 锁具等级与技术开锁收费标准不符合

☐ 白牌锁具价格高于约定价格

☐ 品牌锁具价格高于电商旗舰店同款价格

* 加价区间

☐ ≤1.2倍

☐ 1.2倍<X≤1.5倍

☐ >1.5倍

对应判责结果

☐ 轻费

☐ 中费

☐ 重费

问题描述

请输入问题描述（选填）

取消

确认发起

海星平台-判责工作台

有预约单展示如下字段，若无预约单则顾客信息和履约流程不展示：

- 1、工单信息：同“无忧通”字段
- 2、订单信息：同“无忧通”字段，同步展示预约信息+服务人员信息
- 3、顾客信息：同“无忧通”字段

4、履约流程（展示最新预约单对应的履约信息）

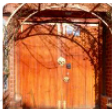
- 顾客下单
 - 师傅签到
 - 服务前检查
 - 1) 开锁前照片
 - 2) 需维修门锁等级
 - 3) 用户是否确认开锁
 - 4) 是否能技术开锁
 - 5) 不能技术开锁原因
 - 6) 新锁照片
 - 7) 新锁等级
 - 拍摄服务后效果
- 直接展示服务后照片

5、客诉信息

如从太平洋进件，则展示，字段和其他现有行业一致

6、判责操作

根据问题场景出对应的判责问题:

0	顾客下单	2025-08-11 22:02:38
0	师傅签到	2025-08-12 09:25:49
0	服务前检查	2025-08-12 09:26:05
	开锁前照片	
	需维修门锁等级	
	B级	
	用户是否确认开锁	
	是	
	是否能技术开锁	
	否	
	不能技术开锁原因	
	超c级无把手电子防盜锁	
	新锁照片	
	新锁等级	
	C级	
0	拍摄服务后效果	2025-08-12 09:27:24
		

	问题场景	判责问题	您认为这笔订单是否存在违规行为？ ☒ 有违规 ☐ 无违规 ☐ 无法判责 1. 是否存在“暴力开锁/过度维修”？ ☒ 是 ☐ 否 对应判罚结果： ☐ 轻责 ☐ 中责 ☒ 重责 2. 是否存在“飞单”？ ☒ 是 ☐ 否 对应判罚结果： ☐ 轻责 ☐ 中责 ☒ 重责 3. 是否存在“不合理收费”？ ☒ 是 ☐ 否 选择不合理收费的细分场景： ☐ 锁具等级与技术开锁收费标准不符合 ☒ 白牌锁具价格高于约定价格 ☐ 品牌锁具价格高于电商旗舰店同款价格 选择加价区间： ☐ ≤1.2倍 ☒ 1.2倍<X≤1.5倍 ☐ >1.5倍 对应判罚结果： ☐ 轻责 ☒ 中责 ☐ 重责
	过度维修/暴力开锁	1.是否存在“暴力开锁/过度维修” 选项：重责、无责、无法判责 2.（若选择无法判责）无法判责的原因是什么？ 选项：无履约信息、履约信息不足、其他 3.补充说明（非必填） 文本框，支持输入	
	飞单	1.是否存在“飞单” 选项：重责、无责、无法判责 2.（若选择无法判责）无法判责的原因是什么？ 选项：无履约信息、履约信息不足、其他 3.补充说明（非必填） 文本框，支持输入	
	不合理收费	1.是否存在“不合理收费” 选项：有责、无责、无法判责 2.（若选择有责）选择细分场景 选项：锁具等级与技术开锁收费标准不符合、白牌锁具价格高于约定价格、品牌锁具价格高于电商旗舰店同款价格 3.（若选择有责）选择加价区间和判责结果 选项：≤1.2倍（轻责）、1.2倍<X≤1.5倍（中责）、>1.5倍（重责） 4.（若选择无法判责）无法判责的原因是什么？ 选项：无履约信息、履约信息不足、其他 5.补充说明（非必填） 文本框，支持输入	
复判（海星平台）	工单判责结果为轻责、中责、重责的，需要进入复判流程 复判问题： 1.这个工单的申诉结果是？ 选项：保持初判结果、更改初判结果 2.（若选择更改初判结果则根据问题场景展示对应判责问题）		

[illegible]

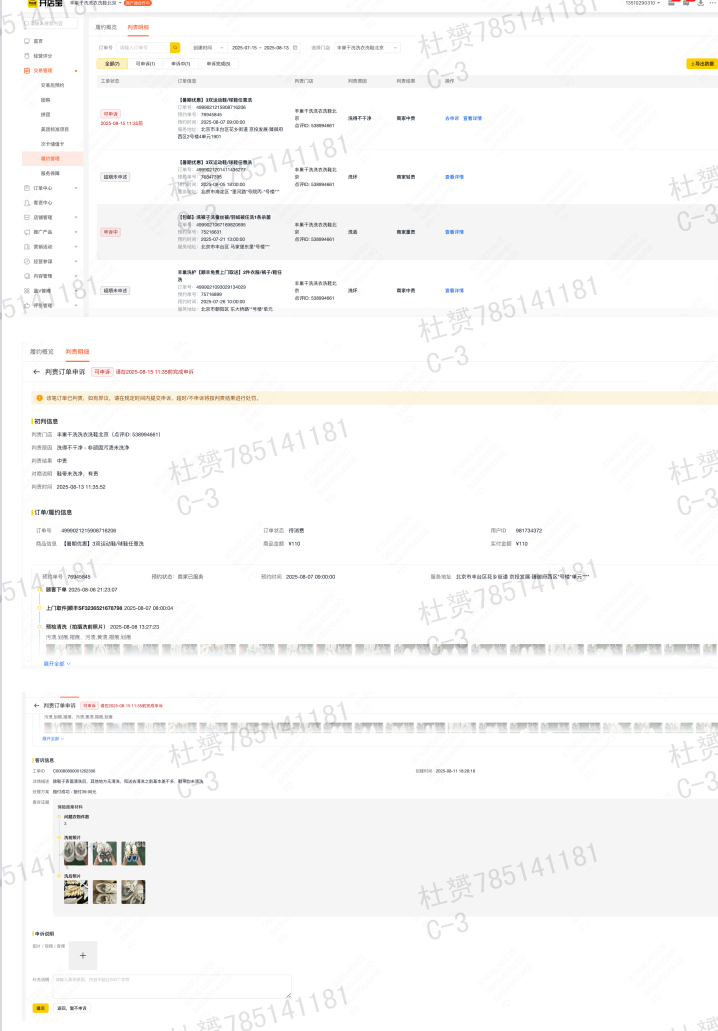
开店宝后台-履约管理-判责
明细

给大连的开锁门店展示判责明细菜单，支持商家申诉
整体复用洗涤现有的功能

差异点如下：

- 1) “判责原因”枚举：过度维修/暴力开锁、不合理收费、飞单
- “不合理收费”下选择的“细分场景”和“加价区间”，不需要对商展示
- 2) 履约信息中的节点和信息，与判责工作台的一致，需适配开锁的场景

商家提交申诉后，进入复判流程



权限

需要新增行业运营权限

分单逻辑

初判：几个人，是谁，平均分单；每天分多少单
复判：几个人，是谁，平均分单； 每天分多少单

历史判责内容展示

展示内容：

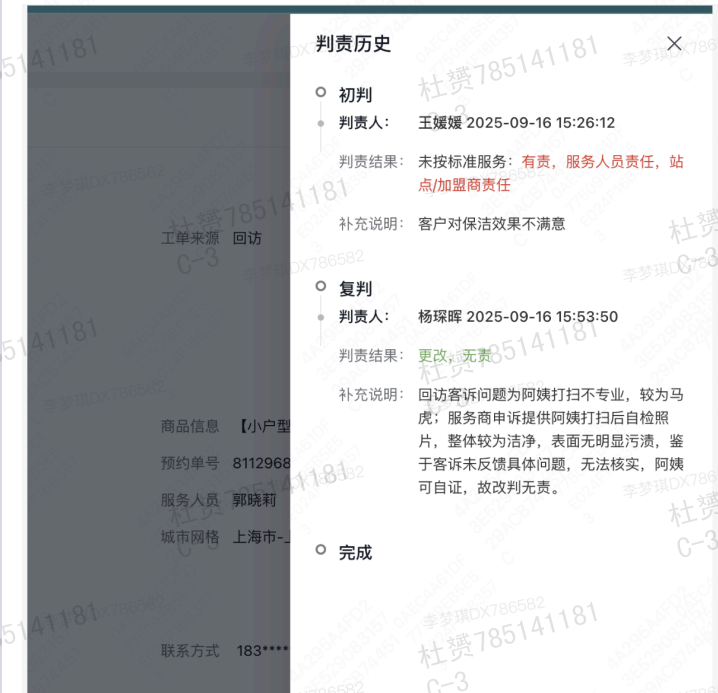
初判：

- 判责人：xxx 判责时间点
- 判责结果：问题场景+判责结果+补充说明

问题场景	判责结果
过度维修/暴力开锁	重责、无责、无法判责
飞单	重责、无责、无法判责
不合理收费	轻责、中责、重责、无责、无法判责

复判：

- 判责人：xxx 判责时间点





- 判责结果：
 - 保持初判结果+补充说明
 - 更改初判结果+复判结果+补充说明

问题场景	判责结果
过度维修/暴力开锁	重责、无责、无法判责
飞单	重责、无责、无法判责
不合理收费	轻责、中责、重责、无责、无法判责

需求背景

开锁换锁行业中履约表现较差，具体表现在高客诉率、售后表现差、平台赔付成本高、舆论风险大等特性；同时，各地监管部门对于生活服务高危行业正在提高对平台的要求；已筛选大连作为试点开锁换锁履约强监控试点城市，验证治理策略的有效性。

需求目标&价值

1. 开锁换锁履约差在商户层面有集中性，通过强监管进行差商管理、降低万单服务率（到25年底下降50%），并降低单均赔付成本
2. 开锁行业若能跑通强监管模式，后期可拓展至管道疏通和防水补漏等高危行业，判责可扩展性强

系统架构

判责业务整体流程



- 判责结果：
 - 保持初判结果+补充说明
 - 更改初判结果+复判结果+补充说明

问题场景	判责结果
过度维修/暴力开锁	重责、无责、无法判责
飞单	重责、无责、无法判责
不合理收费	轻责、中责、重责、无责、无法判责

需求背景

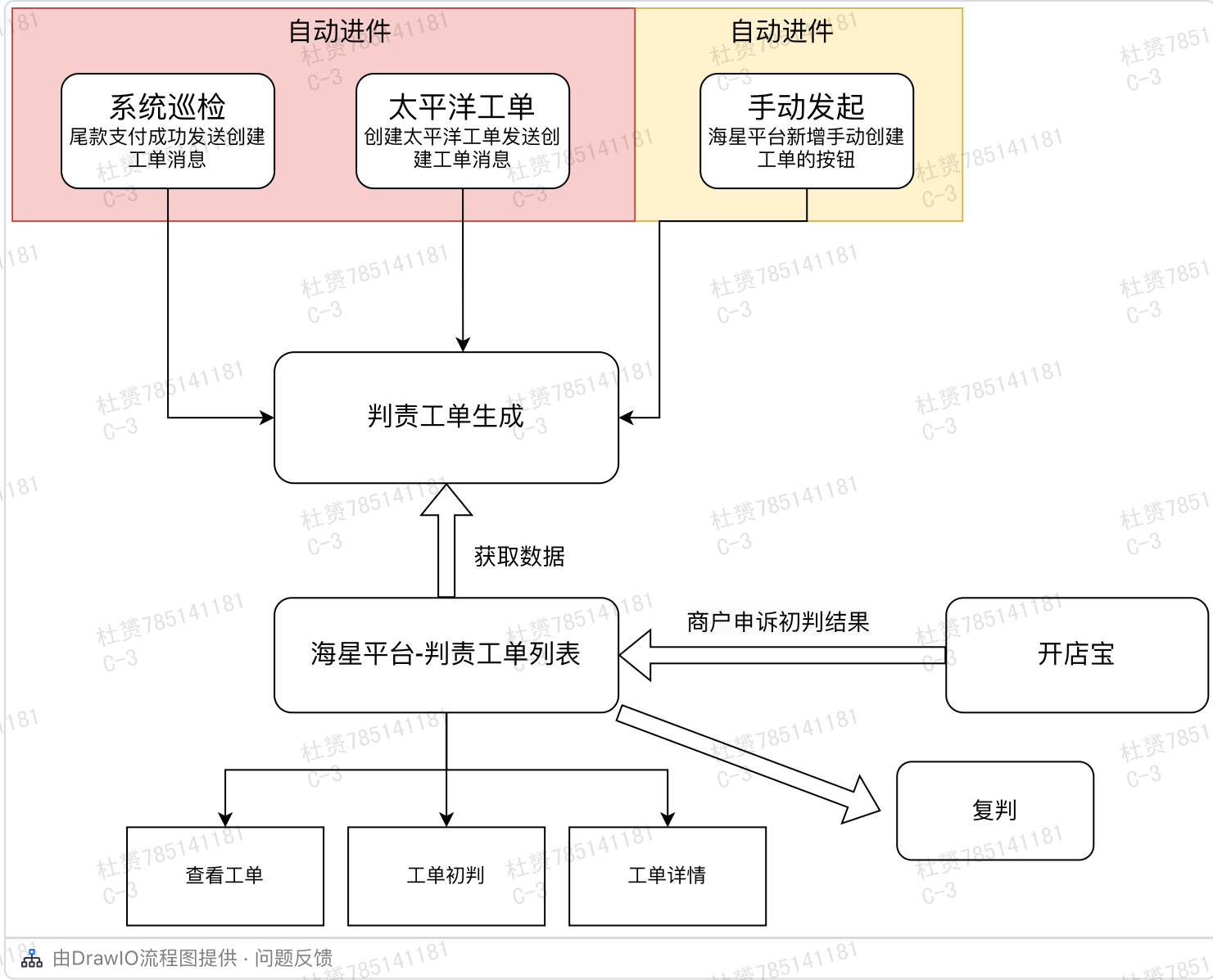
开锁换锁行业中履约表现较差，具体表现在高客诉率、售后表现差、平台赔付成本高、舆论风险大等特性；同时，各地监管部门对于生活服务高危行业正在提高对平台的要求；已筛选大连作为试点开锁换锁履约强监控试点城市，验证治理策略的有效性。

需求目标&价值

1. 开锁换锁履约差在商户层面有集中性，通过强监管进行差商管理、降低万单服务率（到25年底下降50%），并降低单均赔付成本
2. 开锁行业若能跑通强监管模式，后期可拓展至管道疏通和防水补漏等高危行业，判责可扩展性强

系统架构

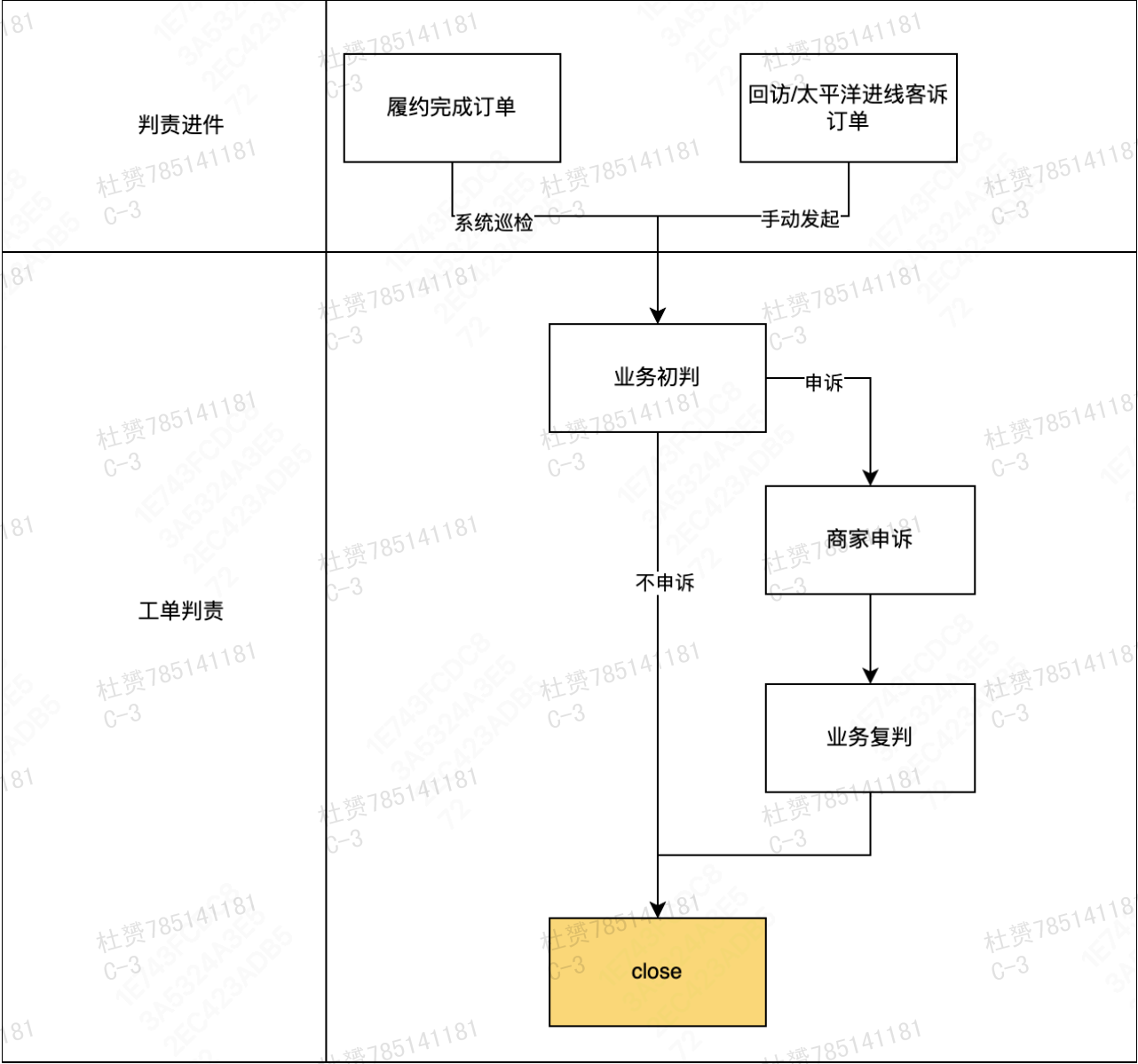
判责业务整体流程



架构说明：

- 进件来源：系统巡检（尾款支付触发）、太平洋工单（客诉进件）、手动发起（运营进件）
- 判责系统核心：进件校验与重复判断、工单池管理与分单逻辑、判责处理与状态流转
- 判责平台：海星平台判责工作台、开店宝后台商户申诉
- 数据存储：判责工单表、工单属性表、处理记录表、工单池表

状态



状态说明：

- 待初判：工单创建后等待分配初判人员
- 初判中：已分配初判人员，正在进行初判
- 商户申诉中：初判有责结果，商户可在48小时内申诉
- 复判中：商户申诉或超时后进入复判流程
- 已完成：判责流程结束

待初判、初判中、待复判、商户申诉中、复判中、已完成

业务流程架构

第一阶段：进件

- 系统巡检进件：订单完成尾款支付 → 校验开锁类目 → 校验大连城市 → 系统巡检进件
- 太平洋工单进件：太平洋工单创建 → FAQ匹配校验 → 太平洋工单进件
- 手动进件：运营手动发起 → 订单校验 → 手动进件
- 重复校验：所有进件都需要通过重复进件校验

第二阶段：分单

● **工单创建**：进件校验通过后创建判责工单

● **实时分单**：工作时间内尝试实时分配初判人员

● **工单池管理**：实时分单失败则进入工单池，通过定时任务分单

第三阶段：判责

● **初判处理**：初判人员进行判责操作

● **申诉流程**：有责结果进入48小时商户申诉期

● **复判处理**：商户申诉或超时后进入复判流程

● **工单完成**：无责/无法判责或复判完成后工单结束

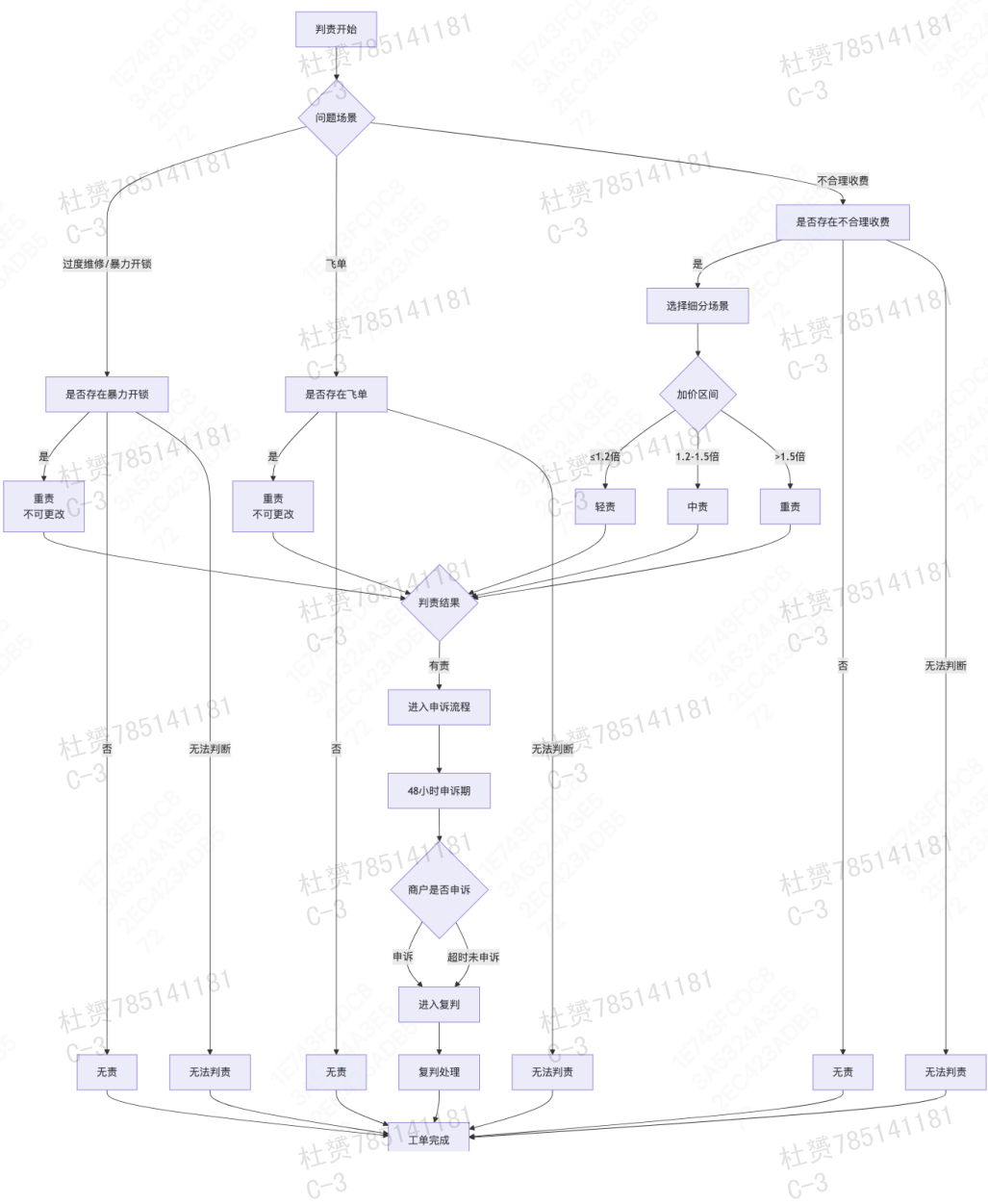
功能描述

进件规则

进件来源	触发条件	备注
尾款订单	1. 订单对应的团单分类为：二级团单类目"开锁" 2. 订单核销门店的城市为大连 3. 订单完成尾款支付后，实时进件	自动进件
太平洋工单	1. 工单关联的门店的城市为大连，门店类目为开锁换锁 2. 工单有关联的订单信息 3. 工单的三级FAQ为：生活服务-线下交易找商户、线下交易找商户、疑似飞单、加价接待、商户价格虚高、消费结果不满意	自动进件
手动发起	运营人员手动输入订单号发起	人工进件

判责规则

违规类型	细分场景	判责结果
暴力开锁/过度维修	-	重责（不可更改）
飞单	-	重责（不可更改）
不合理收费	锁具等级与技术开锁收费标准不符合 白牌锁具价格高于约定价格 品牌锁具价格高于电商旗舰店同款价格	根据加价区间自动判定： ≤1.2倍：轻责 1.2倍< X≤1.5倍：中责 >1.5倍：重责



分单逻辑设计

3.3.1 分单逻辑说明

实时分单流程：

1. 工单创建后，判断是否为工作时间
2. 工作时间内尝试实时分配可用判责人员
3. 分配成功则更新工单状态并推送通知
4. 分配失败则进入工单池等待定时分单

工单池分单流程：

1. 定时任务扫描工单池中的待分单工单
2. 按优先级排序（申诉工单优先，其次按创建时间）
3. 轮询分配给可用人员
4. 批量更新工单状态并推送通知

分单规则配置：

- 初判人员池、复判人员池

- 每日分单限额、轮询算法

3.3.2 分单规则说明

- 初判分单规则：配置初判人员列表，轮询分配，每人每天最大分单数量限制
- 复判分单规则：配置复判人员列表（与初判人员不同），避免分配给同一初判人员
- 申诉优先：商户申诉工单优先分配

数据模型设计

判责工单表 (assess_liability_order)

代码块SQL

```
1 CREATE TABLE `assess_liability_order` (  
2   `assess_liab_order_id` bigint(20) NOT NULL COMMENT '判责工单ID',  
3   `reserve_order_id` bigint(20) DEFAULT NULL COMMENT '预约单ID',  
4   `trade_order_id` varchar(50) NOT NULL COMMENT '交易订单号',  
5   `service_order_id` bigint(20) DEFAULT NULL COMMENT '服务单ID',  
6   `take_order_record_id` bigint(20) DEFAULT NULL COMMENT '接单记录ID',  
7   `business_type` int(11) NOT NULL DEFAULT '5' COMMENT '业务类型（开锁：5）',  
8   `source` int(11) NOT NULL COMMENT '工单来源（3-系统巡检，12-太平洋工单，7-手动发起）',  
9   `status` int(11) NOT NULL DEFAULT '1' COMMENT '状态（1-初判中，2-商户申诉中，3-复判中，9-已完成）',  
10  `assess_result` int(11) DEFAULT '0' COMMENT '判责结果（0-待判责，1-轻责，2-中责，3-重责，4-无责，5-无法判责）',  
11  `service_provider_id` bigint(20) DEFAULT NULL COMMENT '服务商ID',  
12  `shop_id` bigint(20) DEFAULT NULL COMMENT '门店ID',  
13  `tech_name` varchar(100) DEFAULT NULL COMMENT '技师姓名',  
14  `city_id` int(11) DEFAULT NULL COMMENT '城市ID',  
15  `assess_start_time` datetime DEFAULT NULL COMMENT '判责开始时间',  
16  `assess_end_time` datetime DEFAULT NULL COMMENT '判责结束时间',  
17  `add_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',  
18  `update_time` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',  
19  PRIMARY KEY (`assess_liab_order_id`),  
20  KEY `idx_trade_order_id` (`trade_order_id`),  
21  KEY `idx_status` (`status`),  
22  KEY `idx_business_type` (`business_type`),  
23  KEY `idx_city_id` (`city_id`),  
24  KEY `idx_add_time` (`add_time`)  
25 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='判责工单表';  
26
```

判责工单属性表 (assess_liability_order_attr)

代码块SQL

```
1 CREATE TABLE `assess_liability_order_attr` (  
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '自增ID',  
3   `assess_liab_order_id` bigint(20) NOT NULL COMMENT '判责工单ID',  
4   `attr_key` varchar(50) NOT NULL COMMENT '属性键',  
5   `attr_value` text COMMENT '属性值',  
6   `add_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',  
7   `update_time` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',  
8   `is_deleted` tinyint(1) DEFAULT '0' COMMENT '是否删除',  
9   PRIMARY KEY (`id`),  
10  KEY `idx_assess_liab_order_id` (`assess_liab_order_id`),  
11  KEY `idx_attr_key` (`attr_key`)  
12 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='判责工单属性表';  
13
```

商户申诉记录表 (merchant_appeal_record)

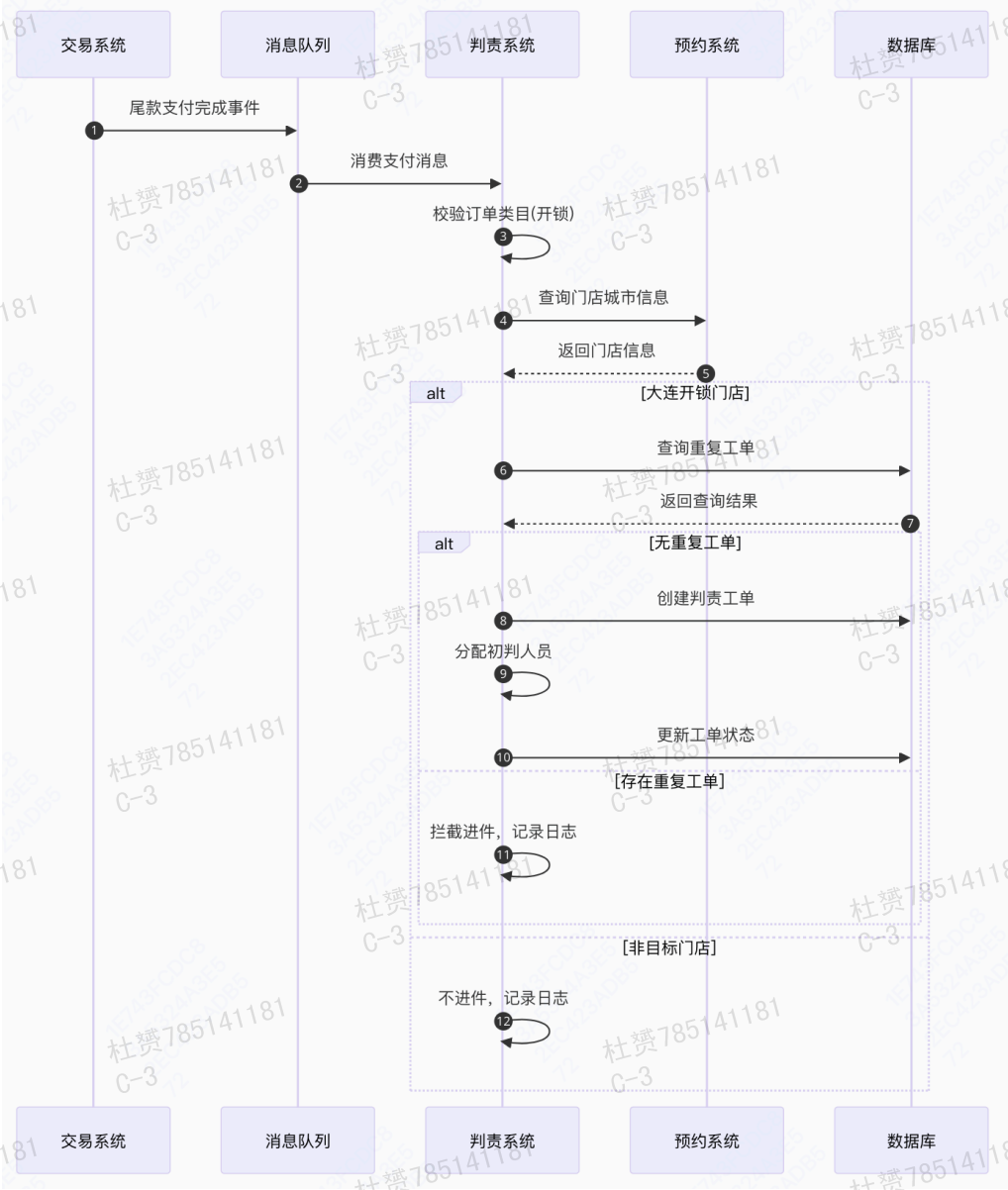
代码块

SQL

```
1 CREATE TABLE `merchant_appeal_record` (  
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '自增ID',  
3   `assess_liab_order_id` bigint(20) NOT NULL COMMENT '判责工单ID',  
4   `service_provider_id` bigint(20) NOT NULL COMMENT '服务商ID',  
5   `appeal_reason` text COMMENT '申诉原因',  
6   `appeal_evidence` text COMMENT '申诉证据',  
7   `appeal_status` int(11) NOT NULL DEFAULT '1' COMMENT '申诉状态（1-待处理，2-处理中，3-申诉成功，4-申诉失败）',  
8   `appeal_result` text COMMENT '申诉处理结果',  
9   `handler_id` bigint(20) DEFAULT NULL COMMENT '处理人员ID',  
10  `handle_time` datetime DEFAULT NULL COMMENT '处理时间',  
11  `add_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',  
12  `update_time` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',  
13  `is_deleted` tinyint(1) DEFAULT '0' COMMENT '是否删除',  
14  PRIMARY KEY (`id`),  
15  KEY `idx_assess_liab_order_id` (`assess_liab_order_id`),  
16  KEY `idx_service_provider_id` (`service_provider_id`)  
17 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='商户申诉记录表';
```

方案

尾款订单



太平洋工单进件



手动进件



初判流程



商户申诉流程



接口文档

海星平台

修改	每次新增行业判责之后就要新增一个if判断，代码可读性、扩展性不强，修改成了策略模式每次就可以继承接口实现策略，可扩展性好
进度	✅
功能	海星平台判责工单列表-筛选器
url	GET:/api/faas/liability/list/filter

request

response

字段	类型	含义
businessType	Integer	业务类型

目前保洁传1，无忧通传2，洗涤传3，维修传4。可以开锁传5

字段	类型	含义
code	Integer	状态
msg	String	信息
data	List<LiabilityListFilterDTO>	筛选器展示数据

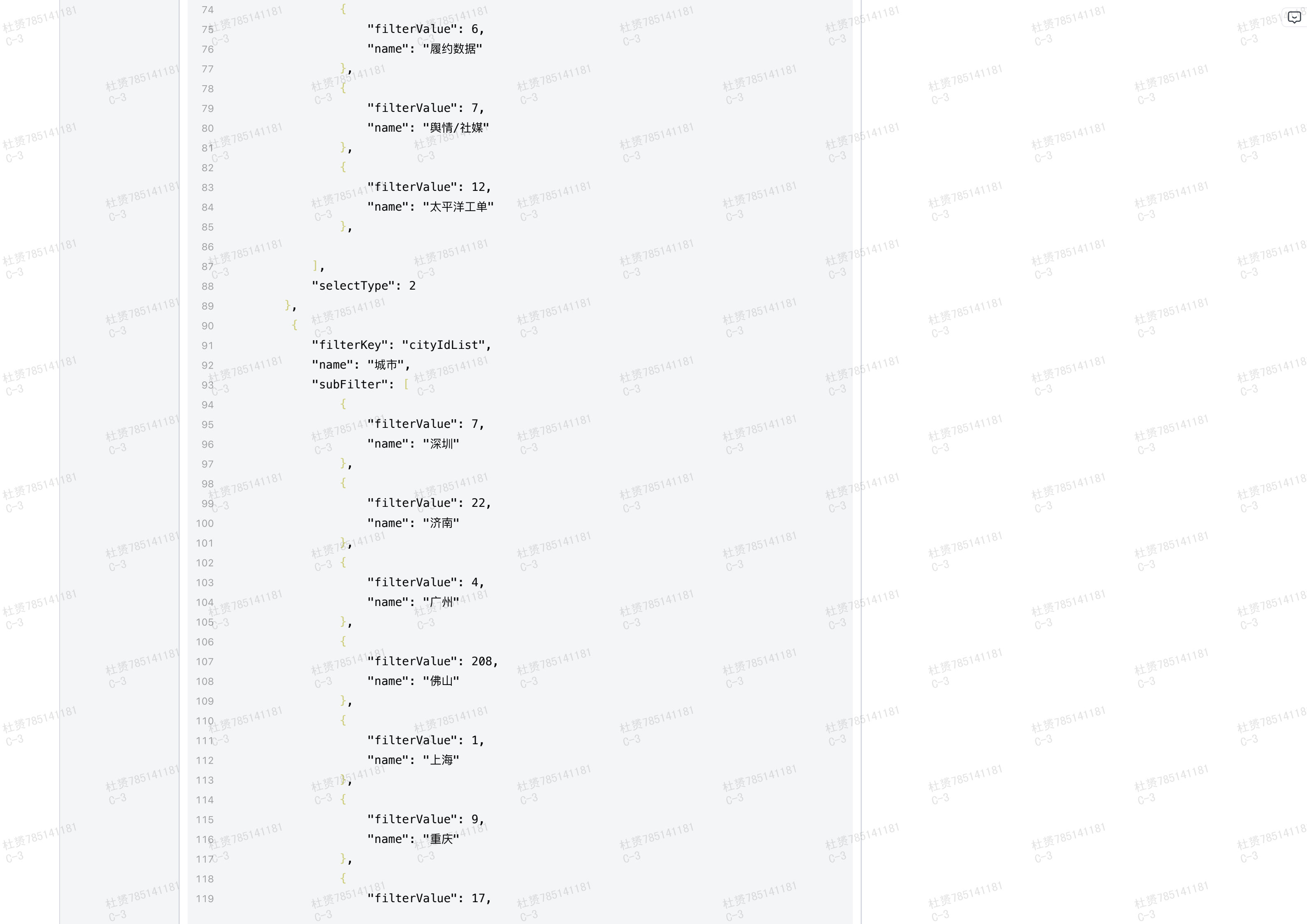
如下

代码块

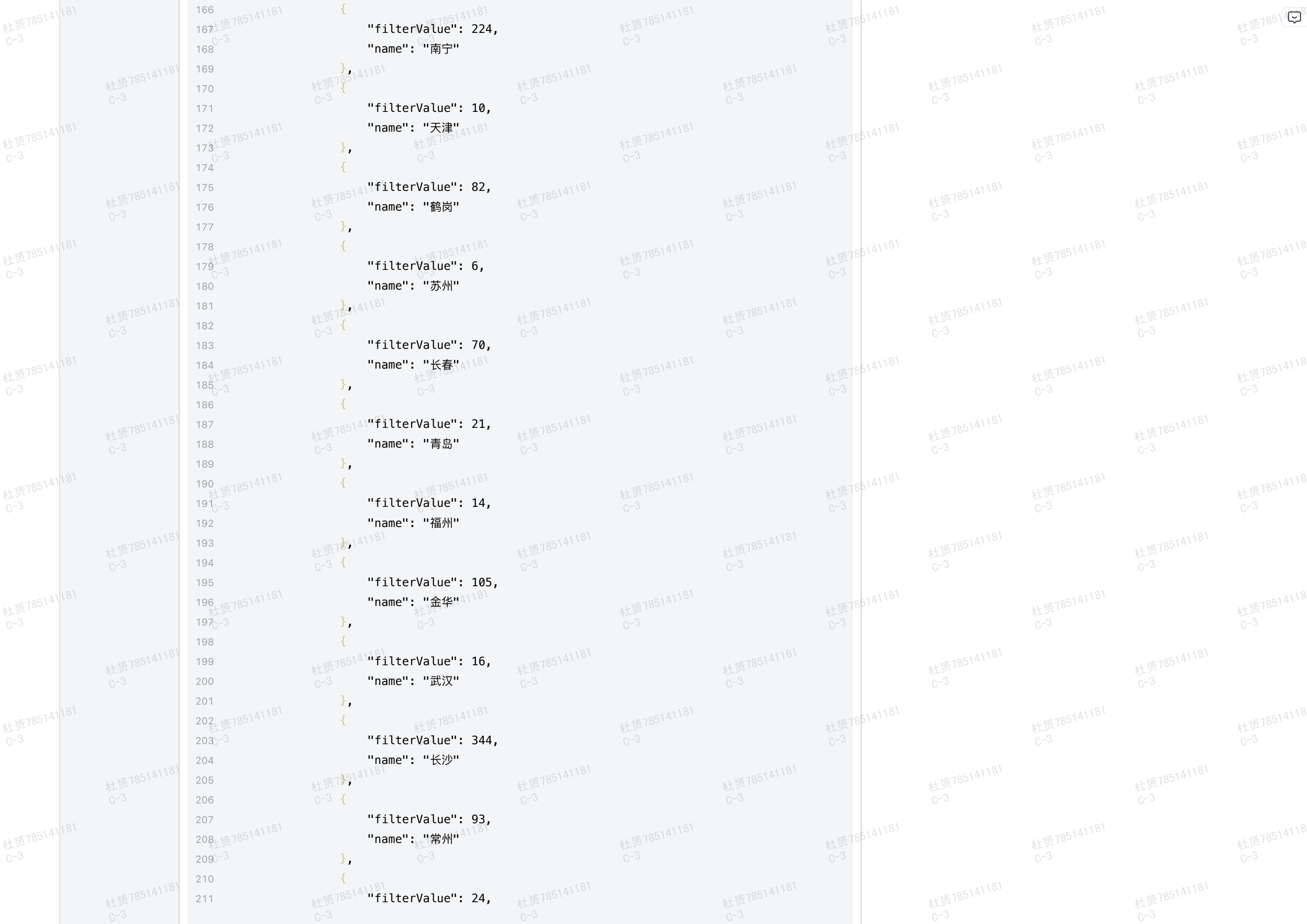
LaTeX

```
1 {
2     "code": 200,
3     "msg": "success",
4     "data": [
5
6         {
7             "filterKey": "searchThingsType",
8             "name": "搜索",
9             "subFilter": [
10
11                 {
12                     "filterValue": 1,
13                     "name": "订单号"
14                 },
15                 {
16                     "filterValue": 2,
17                     "name": "工单号"
18                 }
19             ],
20             "selectType": 1
21         },
22         {
23             "filterKey": "problemScene",
24             "name": "问题场景",
25             "subFilter": [
26
27                 {
28                     "filterValue": 1,
29                     "filterKey": "problemItemsList",
30                     "name": "过度维修/暴力开锁",
31                 }
32             ]
33         }
34     ]
35 }
```

28	"selectType": 2,
29	},
30	{
31	"filterValue": 2,
32	"filterKey": "problemItemsList",
33	"name": "不合理收费",
34	"selectType": 2
35	},
36	{
37	"filterValue": 3,
38	"filterKey": "problemItemsList",
39	"name": "飞单",
40	"selectType": 2
41	},
42	{
43	"filterValue": 4,
44	"filterKey": "problemItemsList",
45	"name": "无违规",
46	"selectType": 2
47	},
48	{
49	"filterValue": 5,
50	"filterKey": "problemItemsList",
51	"name": "无法判责",
52	"selectType": 2
53	}
54	],
55	"selectType": 1
56	},
57	{
58	"filterValue": 0,
59	"filterKey": "workOrderSourceList",
60	"name": "工单来源",
61	"subFilter": [
62	{
63	"filterValue": 3,
64	"name": "系统巡检"
65	},
66	{
67	"filterValue": 4,
68	"name": "回访"
69	},
70	{
71	"filterValue": 5,
72	"name": "差评"
73	},



120	"name": "西安"
121	},
122	{
123	"filterValue": 110,
124	"name": "合肥"
125	},
126	{
127	"filterValue": 219,
128	"name": "东莞"
129	},
130	{
131	"filterValue": 5,
132	"name": "南京"
133	},
134	{
135	"filterValue": 11,
136	"name": "宁波"
137	},
138	{
139	"filterValue": 267,
140	"name": "昆明"
141	},
142	{
143	"filterValue": 8,
144	"name": "成都"
145	},
146	{
147	"filterValue": 13,
148	"name": "无锡"
149	},
150	{
151	"filterValue": 2,
152	"name": "北京"
153	},
154	{
155	"filterValue": 3,
156	"name": "杭州"
157	},
158	{
159	"filterValue": 18,
160	"name": "沈阳"
161	},
162	{
163	"filterValue": 160,
164	"name": "郑州"
165	},



```
212      "name": "石家庄",
213    },
214    {
215      "filterValue": 79,
216      "name": "哈尔滨"
217    },
218    {
219      "filterValue": 35,
220      "name": "太原"
221    },
222    {
223      "filterValue": 258,
224      "name": "贵阳"
225    },
226    {
227      "filterValue": 134,
228      "name": "南昌"
229    },
230    {
231      "filterValue": 252,
232      "name": "雅安"
233    },
234    {
235      "filterValue": 421,
236      "name": "慈溪"
237    }
238  ],
239  "selectType": 2
240 },
241 }
```

进度				
功能	海星平台判责工单列表-状态筛选器：全部、待初判、待复判、初判中、商户申诉中、复判中、已完成			
url	POST:/api/faas/liability/tab			
request	LiabilityTabReq ▼ 代码块 JSON ▼ businessType. 目前保洁传1，无忧通传2，洗涤传3，维修传4。可以开锁传5			
response	<table><tr><th>字段</th><th>类型</th><th>含义</th></tr></table>	字段	类型	含义
字段	类型	含义		

	code	Integer	状态
	msg	String	信息
	data	LiabilityTabDTO	筛选器展示数据
	如下		
	代码块 LaTeX		
进度			
功能	海星平台判责工单列表-判责单列表查询		
url	POST:/api/faas/liability/list		
request	LiabilityListReq 代码块 JSON <pre>1 {"searchThings":null, 2 "searchThingsType":1, 3 "searchStatus":0, 4 "timeType":"1", 5 "searchTime":[1754236800000,1756828799999], 6 "searchStartTime":1754236800000, 7 "searchEndTime":1756828799999, 8 "problemItemsList":null, 9 "districtIdList":null, 10 "searchKey":null, 11 "cityIdList":null, 12 "searchResult":null, 13 "searchTimeType":1, 14 "businessType":3, 15 "pageNo":1, 16 "pageSize":10 17 }</pre> businessType目前保洁传1，无忧通传2，洗涤传3，维修传4。可以开锁传5		
response	字段	类型	含义
	code	Integer	状态

msg	String	信息
data	LiabilityListDTO	工单数据

如下

代码块

Java

```
1  {
2      "code": 200,
3      "msg": "success",
4      "data": {
5          "liabilityOrderList": [
6              {
7                  "id": 1962765384856399894,
8                  "idStr": "1962765384856399894",
9                  "reserveOrderId": 0,
10                 "reserveOrderStatusTag": "已消费",
11                 "tradeOrderId": "4999021395632613194",
12                 "techName": "",
13                 "status": 1,
14                 "statusDesc": "初判中",
15                 "assessResult": "待判责",
16                 "assessResultValue": 0,
17                 "assessAttribute": "加价订单",
18                 "attributeDesc": "-",
19                 "providerName": "莱·洗高质干洗",
20                 "operateJumpUrl": "https://shangmen.meituan.com/haixing-web-static/main/index.html#/liability?assessLiabOrderId=1962765384856399894",
21                 "operate": "去判责",
22                 "operateType": 1,
23                 "assessOrderStartTime": 1756794708000,
24                 "assessSourceCode": 12,
25                 "assessSourceDesc": "太平洋工单",
26                 "operatorName": "于婷",
27                 "assessProblemList": [
28                     "洗坏"
29                 ],
30                 "attrMap": {
31                     "pacific_order_select_result": "[\"咨询\"]",
32                     "pacific_order_faq_id_explain": "轻微损坏",
33                     "problem": "[{\"id\":\"10000002\",\"title\":\"洗坏\", \"type\":1, \"choices\":null}]",
34                     "claim_order_type": "10000002",
35                     "pacific_order_id": "2140807676",
36                     "pacific_order_remark_result": "告知用户商家线下给他修复, 用户接受",
```

37

"pacific_order_faq_id": "214165",

38

"pacific_order_desc": "背景:客户购买了麦齐动物医院和莱·洗高质干洗的服务, 出现了
费问题\n诉求: 客户购买了麦齐动物医院的疫苗服务, 但订单状态未消费, 客户希望得到解决; 客户购买了莱·洗高质干洗的
但衣物未洗干净/洗坏洗丢, 需要退款\n注意事项:",

39

"pacific_order_appeal_material": "
[{\n\"fileName\": \"fb718b34509ea6661372bd3a1e55ff171358299.jpg\", \"id\": \"1605550760\", \"su
\n\"jpg\", \"contentType\": \"image/jpg\", \"filePath\": \"https://p0.meituan.net/csc/fb718b3450
1372bd3a1e55ff171358299.jpg\", \"addTime\": \"1756793331000\"},
\n{\n\"fileName\": \"17cab08bcdcc4451d58cf59df113079b2282405.jpg\", \"id\": \"1605550761\", \"suf
\"jpg\", \"contentType\": \"image/jpg\", \"filePath\": \"https://p1.meituan.net/csc/17cab08bcd
58cf59df113079b2282405.jpg\", \"addTime\": \"1756793331000\"},
\n{\n\"fileName\": \"15f8f0789f41913d22b647abe43874c02562010.jpg\", \"id\": \"1605550762\", \"suf
\"jpg\", \"contentType\": \"image/jpg\", \"filePath\": \"https://p1.meituan.net/csc/15f8f0789f4
2b647abe43874c02562010.jpg\", \"addTime\": \"1756793331000\"}]",

40

"pacific_order_create_time": "1756793331000",

41

"desc": "洗坏"

42

},

43

"businessType": 3,

44

"assessLiabilityObject": {

45

"dpShopId": 1650024669,

46

"shopName": "莱·洗高质干洗"

47

},

48

"currentOperatorName": "于婷",

49

"statusColor": "blue",

50

"showDataExport": true,

51

"showCurrentOperatorName": true

52

},

53

.....

54

],

55

"totalCount": 5286

56

},

57

},

58

}

进度

✓

杜赞785141181
C-3

功能

海星平台判责工单列表-城市列表查询

url

GET:/api/faas/liability/city/list 城市列表

request

字段	类型	含义
businessType	Integer	业务类型

进度	✓											
功能	海星平台判责工单列表-城市列表查询											
url	GET:/api/faas/liability/city/list 城市列表											
request	<table><tr><th>字段</th><th>类型</th><th>含义</th></tr><tr><td>businessType</td><td>Integer</td><td>业务类型</td></tr><tr><td colspan="3"> </td></tr></table>			字段	类型	含义	businessType	Integer	业务类型			
字段	类型	含义										
businessType	Integer	业务类型										

	目前保洁传1，无忧通传2，洗涤传3，维修传4。可以开锁传5		
response	字段	类型	含义
	code	Integer	状态
	msg	String	信息
	data	List<LiabilityListFilterDTO>	城市列表

进度			
功能	海星平台判责工单列表-服务商列表查询		
url	GET:/api/faas/liability/provider/list 服务商		
request	字段	类型	含义
	businessType	Integer	业务类型
	目前保洁传1，无忧通传2，开锁传3，家电维修传4，家电维修传5，家庭维修传6，管道疏通传7		
response	字段	类型	含义
	code	Integer	状态
	msg	String	信息
	data	List<LiabilityListFilterDTO>	城市列表

进度			
功能	海星平台判责工单列表-手动发起工单		
url	POST:/api/faas/liability/list/workorder/submit		
request			
response	字段	类型	含义
	code	Integer	状态
	msg	String	信息

	data	string	添加工单反馈
--	------	--------	--------

进度			
功能	海星平台判责工单列表-判责单详情		
url	GET:/api/faas/order/assign/liability/detail		
request			
response	字段	类型	含义
	code	Integer	状态
	msg	String	信息
	data	string	添加工单反馈

开店宝

判责单进件

系统巡检

太平洋工单

- 工单关联的门店的城市为大连（lion配置目前开锁的城市，可扩展），门店类目为开锁换锁
- 工单有关联的订单信息
- 工单的三级FAQ为：生活服务-线下交易找商户、线下交易找商户、疑似飞单、加价接待、商户价格虚高、消费结果不满意
- 太平洋生成工单的同时向对应topic `csc-pacific-send-daocan-mq` 发送一条消息（包含太平洋工单信息），解析消息并且验证上面的条件；这块使用的**策略模式**，目前判责的行业有保洁、洗涤、无忧通、维修，本次扩开锁，使用策略模式可以以后扩行业更好的扩展

手动--海星平台

排期

9.1月开始 9.22上线

开锁判责系统技术实现详解

一、核心技术架构

1.1 消息队列技术栈

1.1.1 实时进件 - Kafka消息消费

代码块Java

```
1 @FbiMQ(topic = "mptrade.event.domain.status.change.daozong",
2     consumerGroup = "open_the_order_and_pay_final_payment_consumer")
3 public class FinalPaymentOrderConsumer implements FbiMQHandler {
4
5     @Override
6     public FbiMQResponse execute(MessageData messageData) {
7         // 1. 解析交易事件
8         TradeEventDTO tradeEvent = parseTradeEvent(messageData);
9
10        // 2. 校验尾款支付完成
11        if (!isFullPaymentCompleted(tradeEvent)) {
12            return FbiMQResponse.success();
13        }
14
15        // 3. 策略模式匹配业务类型
16        FinalPaymentAssessOrderBizService assessOrderBizService =
17            assessOrderBiz.getAssessOrderBizService(context);
18
19        // 4. 创建判责工单
20        return processAssessOrder();
21    }
22 }
23
```

技术要点:

- **Topic:** mptrade.event.domain.status.change.daozong - 交易状态变更事件
- **消费组:** open_the_order_and_pay_final_payment_consumer - 尾款支付消费组
- **幂等性:** 通过分布式锁保证消息不重复处理

1.1.2 太平洋工单进件 - 策略模式消息处理

代码块Java

```
1 // 太平洋工单消息Topic
2 topic = "csc-pacific-send-daocan-mq"
3
```

```
4 // 开锁业务策略实现
5 @Service("unlockPacificOrderInitAssessOrderProcessor")
6 public class UnlockPacificOrderInitAssessOrderProcessor extends AbstractInitAssessOrderProcessor {
7
8     @Override
9     public boolean accept(CreateOrderAndAttrRequest request) {
10         return AssessSourceEnum.PACIPIC_PLATGFORM.getSource().equals(request.getWorkOrderSource())
11             && AssessBusinessTypeEnum.UNLOCK.getCode() == request.getBusinessType();
12     }
13 }
14
```

1.2 延迟队列技术实现

1.2.1 申诉超时处理

代码块 Java

```
1 // 初判有责后发送延迟消息
2 public TechnicianResp<Boolean> finishFirstAssess(FinishAssessRequest request){
3     // ... 状态流转逻辑
4
5     // 发送延迟消息处理超时未申诉的判责单
6     timeOutAssessOrderProducer.sendDelayMessageForWorkOrderAppealTimeoutPush(assessLiabilityOrderEntity);
7
8     return TechnicianResp.success(result);
9 }
10
```

延迟消息实现机制：

代码块 Java

```
1 @Component
2 public class TimeOutAssessOrderProducer {
3
4     public void sendDelayMessageForWorkOrderAppealTimeoutPush(AssessLiabilityOrderEntity entity) {
5         // 48小时延迟消息
6         long delayTime = 48 * 60 * 60 * 1000L; // 48小时
7
8         DelayMessage delayMessage = DelayMessage.builder()
9             .topic("assess_order_appeal_timeout")
10            .delayTime(delayTime)
11            .payload(JsonUtils.toJson(entity))
12            .build();
13    }
```

```
14         delayMessageProducer.send(delayMessage);
15     }
16 }
17
18 // 延迟消息消费者
19 @FbiMQ(topic = "assess_order_appeal_timeout")
20 public class AppealTimeoutConsumer implements FbiMQHandler {
21
22     @Override
23     public FbiMQResponse execute(MessageData messageData) {
24         // 1. 检查工单状态是否仍为申诉中
25         // 2. 如果超时未申诉, 自动流转到复判
26         // 3. 分配复判人员
27         return autoTransferToRetrial(assessLiabilityOrderEntity);
28     }
29 }
30
```

1.3 分布式锁技术

1.3.1 Redis分布式锁防重复处理

代码块Java

```
1 @Component
2 public class RedisLockUtils {
3
4     public <T> T tryLock(String key, Func2<T> func2, int timeout) {
5         Lock lock = distributedLockManager.getReentrantLock(key, DEFAULT_EXPIRY_TIME_MILLIS);
6
7         try {
8             if (lock.tryLock(timeout, TimeUnit.SECONDS)) {
9                 try {
10                     return func2.invoke();
11                 } finally {
12                     lock.unlock();
13                 }
14             } else {
15                 throw new RuntimeException("存在重复提交, 请稍后再试");
16             }
17         } catch (InterruptedException e) {
18             Thread.currentThread().interrupt();
19         }
20         return null;
21     }
22 }
```

应用场景：

代码块

```
1 // 1. 工单状态更新锁
2 String key = DOORTOD00R_FINSH_ASSESS_ORDER + assessLiabOrderId;
3 redisLockUtils.tryLock(key, () -> {
4     // 更新工单状态的原子操作
5     updateAssessOrderStatus();
6 }, 5);
7
8 // 2. 重复进件检查锁
9 String bizKey = RedisKeyUtils.getUnlockKey(businessType, orderId, attrs);
10 boolean isDuplicate = duplicateOrderCheck.duplicateOrderCheck(bizKey);
11
```

1.4 重复进件控制技术

1.4.1 业务唯一键生成策略

代码块

```
1 public class RedisKeyUtils {
2
3     public static String getUnlockKey(String businessType, Long orderId, Map<String, String> attrs) {
4         // 构建业务唯一键：业务类型_订单ID_问题场景
5         String problemScene = extractProblemScene(attrs);
6         return String.format("assess_order:%s:%s:%s", businessType, orderId, problemScene);
7     }
8
9     private static String extractProblemScene(Map<String, String> attrs) {
10        // 从FAQ映射到问题场景
11        String faqId = attrs.get(AssessAttrKeyEnum.PACIFIC_ORDER_FAQ_ID.getAttrKey());
12        return mapFaqToProblemScene(faqId);
13    }
14 }
15
```

1.4.2 重复进件处理逻辑

代码块

```
1 @Component
2 public class DuplicateOrderCheck {
```

```
3
4 public boolean duplicateOrderCheck(String bizKey) {
5     // Redis SET NX 操作, 过期时间24小时
6     String lockKey = "duplicate_check:" + bizKey;
7     return redisTemplate.opsForValue().setIfAbsent(lockKey, "1", 24, TimeUnit.HOURS);
8 }
9
10 public void handleDuplicateOrder(String bizKey) {
11     // 重复进件时, 给原工单打标签
12     AssessLiabilityOrderEntity existingOrder = findExistingOrder(bizKey);
13     if (existingOrder != null) {
14         // 添加"太平洋工单" 标签
15         addPacificOrderTag(existingOrder);
16     }
17 }
18 }
19 }
```

二、数据库技术实现

2.1 分库分表策略

代码块 Java

```
1 // 判责工单表按业务类型分表
2 @Table(name = "assess_liability_order")
3 @ShardingKey("business_type")
4 public class AssessLiabilityOrderEntity {
5
6     @Id
7     private Long assessLiabOrderId; // 雪花算法生成
8
9     @ShardingColumn
10    private Integer businessType; // 分表键
11
12    // 其他字段...
13 }
14
```

2.2 数据库事务管理

代码块 Java

```
1 @Component
2 public class TransactionUtil {
3
4     // ...
5 }
```

```
3
4 @Transactional(rollbackFor = Exception.class)
5 public void transaction(Runnable runnable) {
6     try {
7         // 强制使用主库
8         ZebraForceMasterHelper.forceMasterInLocalContext();
9         runnable.run();
10    } catch (Exception e) {
11        log.error("Transaction execution failed", e);
12        throw e;
13    }
14 }
15 }
16 }
```

2.3 读写分离配置

```
代码块
1 // 强制主库读取
2 private void forceMasterDatabase() {
3     ZebraForceMasterHelper.forceMasterInLocalContext();
4 }
5
6 // 查询判责单时强制主库
7 AssessLiabilityOrderEntity entity = assessLiabilityOrderRepository.findById(orderId);
8
```

三、缓存技术实现

3.1 Redis缓存策略

```
代码块
1 @Component
2 public class AssessOrderCacheService {
3
4     // 工单信息缓存, 过期时间30分钟
5     public AssessLiabilityOrderEntity getAssessOrderFromCache(Long orderId) {
6         String cacheKey = "assess_order:" + orderId;
7         String cached = redisTemplate.opsForValue().get(cacheKey);
8
9         if (StringUtils.isEmpty(cached)) {
10             return JsonUtils.fromJson(cached, AssessLiabilityOrderEntity.class);
11         }
12     }
13 }
```

```
12 // 缓存未命中, 查询数据库
13 AssessLiabilityOrderEntity entity = queryFromDatabase(orderId);
14 if(entity != null) {
15     redisTemplate.opsForValue().set(cacheKey, JsonUtils.toJson(entity), 30, TimeUnit.MINUTES);
16 }
17
18
19 return entity;
20 }
21 }
22
```

3.2 分布式Session管理

代码块

```
1 // 判责人员工作状态缓存
2 public class OperatorStatusCache {
3
4     public void updateOperatorWorkload(String operatorId, int workload) {
5         String key = "operator_workload:" + operatorId;
6         redisTemplate.opsForValue().set(key, String.valueOf(workload), 1, TimeUnit.HOURS);
7     }
8
9     public int getOperatorWorkload(String operatorId) {
10         String key = "operator_workload:" + operatorId;
11         String workload = redisTemplate.opsForValue().get(key);
12         return StringUtils.isEmpty(workload) ? Integer.parseInt(workload) : 0;
13     }
14 }
15
```

四、分单调度技术

4.1 实时分单算法

代码块

Java

```
1 @Component
2 public class RealTimeDistributionService {
3
4     public boolean distributeOrderRealTime(AssessLiabilityOrderEntity order) {
5         // 1. 获取可用判责人员列表
6         List<OperatorInfo> availableOperators = getAvailableOperators(order.getBusinessType());
7     }
8 }
```

```
8 // 2. 轮询算法分配
9 OperatorInfo selectedOperator = roundRobinSelect(availableOperators);
10
11 // 3. 原子性分配操作
12 return atomicAssignOrder(order.getAssessLiabOrderId(), selectedOperator.getOperatorId());
13 }
14
15 private OperatorInfo roundRobinSelect(List<OperatorInfo> operators) {
16     // Redis计数器实现轮询
17     String counterKey = "operator_round_robin_counter";
18     Long counter = redisTemplate.opsForValue().increment(counterKey);
19     int index = (int) (counter % operators.size());
20     return operators.get(index);
21 }
22 }
23
```

4.2 工单池定时调度

代码块

Java

```
1 @Component
2 @Slf4j
3 public class OrderPoolScheduler {
4
5     @Scheduled(fixedDelay = 60000) // 每分钟执行一次
6     public void distributeOrdersFromPool() {
7         try {
8             // 1. 从工单池获取待分配工单
9             List<AssessLiabilityOrderEntity> pendingOrders = getOrdersFromPool();
10
11             // 2. 按优先级排序（申诉工单优先）
12             pendingOrders.sort(this::compareOrderPriority);
13
14             // 3. 批量分配
15             batchDistributeOrders(pendingOrders);
16
17         } catch (Exception e) {
18             log.error("Order pool distribution failed", e);
19         }
20     }
21
22     private int compareOrderPriority(AssessLiabilityOrderEntity o1, AssessLiabilityOrderEntity o2) {
23         // 申诉工单优先级最高
24         if (LiabilityStatusEnum.MERCHANT_APPEAL.getStatus().equals(o1.getStatus())) {
25             return -1;
```

```
26     }
27     if (LiabilityStatusEnum.MERCHANT_APEAL.getStatus().equals(o2.getStatus())) {
28         return 1;
29     }
30     // 其次按创建时间排序
31     return o1.getAddTime().compareTo(o2.getAddTime());
32 }
33 }
34
```

五、监控告警技术

5.1 业务监控指标

代码块

```
1 @Component
2 public class AssessOrderMonitor {
3
4     // Cat监控埋点
5     public void recordAssessOrderMetrics(String operation, boolean success, long duration) {
6         Transaction transaction = Cat.newTransaction("AssessOrder", operation);
7         try {
8             if (success) {
9                 Cat.logEvent("AssessOrder.Success", operation);
10            } else {
11                Cat.logEvent("AssessOrder.Fail", operation);
12            }
13
14            // 记录耗时
15            Cat.logMetricForDuration("AssessOrder.Duration." + operation, duration);
16
17        } finally {
18            transaction.setStatus(success ? Transaction.SUCCESS : "FAIL");
19            transaction.complete();
20        }
21    }
22
23    // 业务指标监控
24    @Scheduled(fixedRate = 300000) // 5分钟
25    public void reportBusinessMetrics() {
26        // 统计各状态工单数量
27        Map<Integer, Long> statusCount = assessLiabilityOrderRepository.countByStatus();
28
29        statusCount.forEach((status, count) -> {
30            Cat.logMetricForCount("AssessOrder.Status." + status, count.intValue());
31        });
32    }
33 }
```

```
31    });
32
33    // 统计超时工单
34    long timeoutCount = countTimeoutOrders();
35    Cat.logMetricForCount("AssessOrder.Timeout", (int) timeoutCount);
36 }
37
38
```

5.2 告警配置

代码块Java

```
1  @Component
2  public class AlertService {
3
4      public void checkAndAlert() {
5          // 1. 检查初判超时工单
6          long firstAssessTimeout = countOrdersByStatusAndTimeout(
7              LiabilityStatusEnum.FIRST_ASSESS.getStatus(), 24 * 60 * 60 * 1000L);
8
9          if (firstAssessTimeout > 10) {
10              sendAlert("初判超时工单数量异常", "超时工单数量: " + firstAssessTimeout);
11          }
12
13          // 2. 检查复判超时工单
14          long secondAssessTimeout = countOrdersByStatusAndTimeout(
15              LiabilityStatusEnum.SECOND_ASSESS.getStatus(), 48 * 60 * 60 * 1000L);
16
17          if (secondAssessTimeout > 5) {
18              sendAlert("复判超时工单数量异常", "超时工单数量: " + secondAssessTimeout);
19          }
20
21          // 3. 检查系统巡检进件成功率
22          double successRate = calculateIncomingSuccessRate();
23          if (successRate < 0.95) {
24              sendAlert("系统巡检进件成功率异常", "成功率: " + successRate);
25          }
26      }
27  }
28
```

六、性能优化技术

6.1 异步处理

代码块

Java

```
1 @Component
2 public class AsyncAssessOrderProcessor {
3
4     @Async("assessOrderExecutor")
5     public CompletableFuture<Void> processAssessOrderAsync(AssessLiabilityOrderEntity order) {
6         try {
7             // 1. 异步推送消息给商户
8             messagePushService.pushToMerchant(order);
9
10            // 2. 异步更新统计数据
11            statisticsService.updateAssessOrderStats(order);
12
13            // 3. 异步同步数据到数仓
14            dataWarehouseService.syncAssessOrderData(order);
15
16        } catch (Exception e) {
17            log.error("Async process assess order failed", e);
18        }
19
20        return CompletableFuture.completedFuture(null);
21    }
22 }
23
24 // 线程池配置
25 @Configuration
26 public class AsyncConfig {
27
28     @Bean("assessOrderExecutor")
29     public Executor assessOrderExecutor() {
30         ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
31         executor.setCorePoolSize(10);
32         executor.setMaxPoolSize(20);
33         executor.setQueueCapacity(200);
34         executor.setThreadNamePrefix("assess-order-");
35         executor.initialize();
36         return executor;
37     }
38 }
39
```

6.2 批量处理优化

代码块

Java

```
1 @Component
2 public class BatchProcessService {
3
4     @Transactional
5     public void batchUpdateOrderStatus(List<Long> orderIds, Integer toStatus) {
6         // 批量更新, 每批100条
7         int batchSize = 100;
8         for (int i = 0; i < orderIds.size(); i += batchSize) {
9             int endIndex = Math.min(i + batchSize, orderIds.size());
10            List<Long> batchIds = orderIds.subList(i, endIndex);
11
12            assessLiabilityOrderRepository.batchUpdateStatus(batchIds, toStatus);
13        }
14    }
15
16    public void batchInsertOrderLogs(List<AssessLiabilityOrderLogEntity> logs) {
17        // MyBatis批量插入
18        assessLiabilityOrderLogRepository.batchInsert(logs);
19    }
20 }
21
```

七、技术架构总结

7.1 核心技术栈

- 消息队列: Kafka + 自研FbiMQ框架
- 延迟队列: 基于Kafka的延迟消息机制
- 分布式锁: Redis + Redisson
- 缓存: Redis集群
- 数据库: MySQL分库分表 + Zebra读写分离
- 监控: Cat监控 + 自研告警系统
- 异步处理: Spring @Async + 自定义线程池

7.2 关键设计模式

- 策略模式: 不同业务类型的判责逻辑
- 模板方法模式: 抽象判责处理流程
- 观察者模式: 状态变更事件通知
- 工厂模式: 业务处理器创建

7.3 高可用保障

- **幂等性:** 分布式锁 + 业务唯一键
- **容错性:** 异常重试+ 降级处理
- **监控告警:** 实时监控 + 多维度告警
- **数据一致性:** 分布式事务 + 最终一致性

这套技术架构确保了开锁判责系统的高性能、高可用和可扩展性，能够支撑大规模的判责业务处理需求。

基于你提供的开锁判责方案设计文档，我来为你整理面试可能问到的问题及参考答案。这些问题会从业务理解、技术实现、系统设计等多个维度来准备。

面试相关

一、业务相关问题

Q1: 请简单介绍一下这个开锁判责项目的背景和目标

A: 这个项目是针对开锁换锁行业履约表现差的问题而设计的。主要背景包括：

- 开锁行业客诉率高（大连6.8% vs 全国平均）
- 赔付成本高（大连单均1.6元 vs 全国1.3元）
- 舆论风险大，影响平台商誉
- 监管部门对高危行业要求趋严

目标是通过强监管模式：

- 降低万单服务率（到25年底下降50%）
- 降低单均赔付成本
- 验证治理策略有效性，为后续拓展到管道疏通、防水补漏等行业做准备

Q2: 判责的标准和分类有哪些？

A: 判责主要分为三类违规类型：

1. **暴力开锁/过度维修：**直接判定为**重责**
2. **飞单：**直接判定为**重责**
3. **不合理收费：**根据加价倍数细分：
 - ≤ 1.2 倍：轻责
 - $1.2\text{倍} < X \leq 1.5$ 倍：中责
 - > 1.5 倍：重责

此外还有"无违规"和"无法判责"两种结果。

Q3: 进件的来源有哪些？触发条件是什么？

A: 进件来源有三种：

1. 系统巡检（自动）：

- 订单类目为"开锁"
- 门店城市为大连
- 完成尾款支付后实时触发

2. 太平洋工单（自动）：

- 工单关联大连开锁门店
- 有关联订单信息
- 三级FAQ匹配特定关键词

3. 手动发起（人工）：

- 运营人员在海星平台手动输入订单号发起

二、技术实现相关问题

Q4: 系统巡检的尾款支付消息处理流程是怎样的？

A: 系统巡检通过监听交易侧的尾款支付消息实现：

代码块

Java

1

// 核心流程在FinalPaymentOrderConsumer中

2

1. 监听topic: "mptrade.event.domain.status.change.daozong"

3

2. 验证消息有效性（业务编码、订单ID等）

4

3. 判断是否完成尾款支付（实际支付金额=订单总金额）

5

4. 查询订单信息，构建FinalPaymentContext

6

5. 通过策略模式找到对应的业务处理服务

7

6. 创建判责工单

8

Q5: 策略模式在这个项目中是如何应用的？

A: 策略模式主要应用在两个方面：

1. 尾款支付业务处理：

代码块

Java

1

public interface FinalPaymentAssessOrderBizService {

2

boolean accept(FinalPaymentContext context); // 判断是否支持该业务

3

Integer getAssessOrderBizType(FinalPaymentContext context);

4

CreateOrderAndAttrRequest buildCreateAssessOrderRequest(FinalPaymentContext context);

5

}

1. 不同行业的扩展:

- 保洁、洗涤、无忧通、维修、开锁等行业都有各自的实现类
- 通过 `accept()` 方法自动匹配对应的业务处理器
- 新增行业时只需新增实现类，无需修改原有代码

Q6: 如何防止重复进件?

A: 重复进件判断逻辑:

1. **系统巡检重复判断**：根据订单ID和问题场景判断
2. **太平洋工单重复处理**：如果订单已有相同问题场景的工单，拦截进件
3. **手动发起校验**：点击"确认发起"时校验是否存在相同问题场景的工单

实现方式：在进件时查询数据库，判断对应订单是否已存在相同问题场景的工单。

Q7: 判责工单的状态流转是怎样的?

A: 判责工单状态包括:

- **待初判：**新创建的工单
- **初判中：**初判人员正在处理
- **待复判：**初判完成，需要复判
- **商户申诉中：**商户对判责结果有异议
- **复判中：**复判人员正在处理
- **已完成：**最终判责结果确定

状态流转支持正向和逆向流程，商户可以申诉，运营人员可以手动调整。

三、系统设计相关问题

Q8: 这个系统的整体架构是怎样的?

A: 系统采用分层架构设计:



9	数据存储层		
10	- MySQL(工单数据)	- Redis(缓存)	
11			
12	外部系统对接		
13	- 交易系统	- 太平洋工单	- 门店系统
14			
15			

核心模块：

- **assessmq**：消息处理模块，处理尾款支付和太平洋工单消息
- **aibiz**：判责中台核心，提供工单创建、查询、判责等能力
- **serviceprovider**：业务服务提供层，封装各行业的特殊逻辑

Q9: 如何设计一个可扩展的判责系统，支持未来更多行业？

A: 关键设计点：

1. **策略模式**：各行业实现统一的业务接口
2. **配置化**：城市、类目、FAQ等通过配置管理
3. **模板方法**：抽象类定义通用流程，子类实现特殊逻辑
4. **枚举标准化**：业务类型、问题场景、判责结果等都用枚举维护
5. **数据库设计**：预留扩展字段，支持不同行业的特殊属性

代码示例：

代码块

```
1 //通过Lion配置支持的城市和类目
2 @Value("${liability.cities:7}") // 默认支持大连
3 private String supportCities;
4
5 @Value("${liability.categories:826}") // 开锁类目
6 private String supportCategories;
7
```

Java

Q10: 如果系统处理量大，如何优化性能？

A: 性能优化方案：

1. **消息处理优化**：
 - 批量处理消息
 - 异步创建工单
 - 失败重试机制

2. 查询优化：

- 索引优化（订单ID、状态、时间等）
- 分页查询
- 缓存热点数据

3. 系统解耦：

- 消息队列削峰填谷
- 异步处理非关键路径
- 读写分离

4. 资源隔离：

- 不同行业使用不同队列
- 核心接口限流保护
- 降级预案（手动进件兜底）

四、项目经验相关问题

Q11: 在这个项目中你承担了什么角色？遇到过什么挑战？

A:（根据实习生的身份，可以这样回答）

承担的角色：

- 参与需求评审和技术方案讨论
- 负责部分模块的代码开发（如消息处理、工单创建等）
- 协助进行测试和bug修复
- 整理项目文档和接口说明

遇到的挑战：

- 业务复杂性：** 开锁行业的特殊规则和判责标准需要深入理解
- 系统兼容性：** 与多个外部系统对接，接口协议不一致
- 数据一致性：** 确保各系统间的数据同步和状态一致

解决方法：

- 主动与产品、运营沟通，理解业务背景
- 仔细阅读接口文档，提前识别风险点
- 增加数据校验和异常处理机制

Q12: 从这个项目中学到了什么？

A: 主要收获：

- 业务理解能力：** 学会了如何从业务角度思考技术方案

2. **系统设计思维**：理解了高内聚、低耦合的设计原则
3. **编码规范**：掌握了企业级的代码规范和最佳实践
4. **团队协作**：体验了跨团队沟通的重要性
5. **问题定位**：提升了通过日志、监控快速定位问题的能力

Q13: 如果让你独立设计一个类似的判责系统，你会怎么做？

A: 设计思路：

1. 需求分析阶段：

- 深入了解业务流程和规则
- 识别核心痛点和关键指标
- 与业务方确认验收标准

2. 技术设计阶段：

- 选择合适的技术栈和架构模式
- 设计可扩展的数据模型
- 制定接口规范和错误码标准

3. 核心功能实现：

- 配置化的规则引擎
- 灵活的工单状态机
- 统一的消息处理框架

4. 质量保障：

- 单元测试覆盖率>80%
- 集成测试覆盖核心流程
- 监控告警机制完善

5. 性能优化：

- 支持水平扩展
- 关键接口限流
- 数据库读写分离

Q14: 对实习生来说，参与这个项目最大的价值是什么？

A: 最大价值：

1. **真实业务场景**：接触到企业级复杂业务系统的完整生命周期
2. **技术栈实践**：深度使用Spring、MyBatis、Redis、MQ等主流技术
3. **软技能提升**：学会与产品、测试、运维等多方协作
4. **业务思维培养**：理解技术如何服务于业务目标

5. 代码质量意识：在大规模代码库中保持代码质量的重要性

Q15: 如果面试官让你提问，你会问什么？

A: 建议的提问：

- 团队技术氛围**：团队如何做技术选型和代码review？
- 成长路径**：实习生转正的标准和成长路径是怎样的？
- 业务挑战**：目前团队在技术上最大的挑战是什么？
- 学习资源**：公司有哪些内部学习资源可以推荐？

面试准备建议

1. 技术准备

- 重点掌握**：策略模式、消息队列、状态机设计
- 代码熟悉**：能够讲解核心代码逻辑，特别是消息处理和工单创建部分
- 异常处理**：了解常见的异常情况如何处理

2. 业务理解

- 数据准备**：记住关键数据（大连客诉率6.8%，单均赔付1.6元等）
- 流程图**：能够在纸上画出判责流程和状态流转
- 行业对比**：理解开锁行业与其他行业的差异

3. 表达技巧

- STAR法则**：描述项目经验时用Situation-Task-Action-Result结构
- 量化成果**：尽量用数据说明项目价值
- 问题反思**：诚实地说出遇到的问题和学到的教训

4. 注意事项

- 不要夸大**：作为实习生，重点展示学习能力和成长潜力
- 保持谦逊**：承认还有很多需要学习的地方
- 积极主动**：表达对技术和业务的持续学习热情

祝你面试顺利！记住，面试官更看重你的潜力和学习态度，而不是已经掌握了多少技能。保持自信，展现出你对技术的热情和对业务的理解就好。